

# Celery at the center of your pipelines

168: LinkedIn | The "Green-IT" Revolution in Task Scheduling

## The context

Did you know that task brokers like Celery can consume up to 10x more energy than a lightweight alternative? wpipe is leading the charge in sustainable software with its <50MB RAM footprint.

## Why engineers shift to wpipe

- Pipelines are Python: versionable, testable, reviewable in Git.
- State persisted to SQLite WAL at every step; resume, don't restart.
- <50MB RAM. No broker, no daemon, no metadata server.

### Code-first

Pipelines are Python: versionable, testable, reviewable in Git.

### Checkpoints

State persisted to SQLite WAL at every step; resume, don't restart.

### Independently lean

<50MB RAM. No broker, no daemon, no metadata server.

## What it looks like

```
from wpipe import Pipeline, step

@step(name="job", retry_count=3, retry_delay=1)
def job(data):
    return {"result": worker(data["task"])}

pipe = Pipeline(pipeline_name="brokerless")
pipe.set_steps([job])

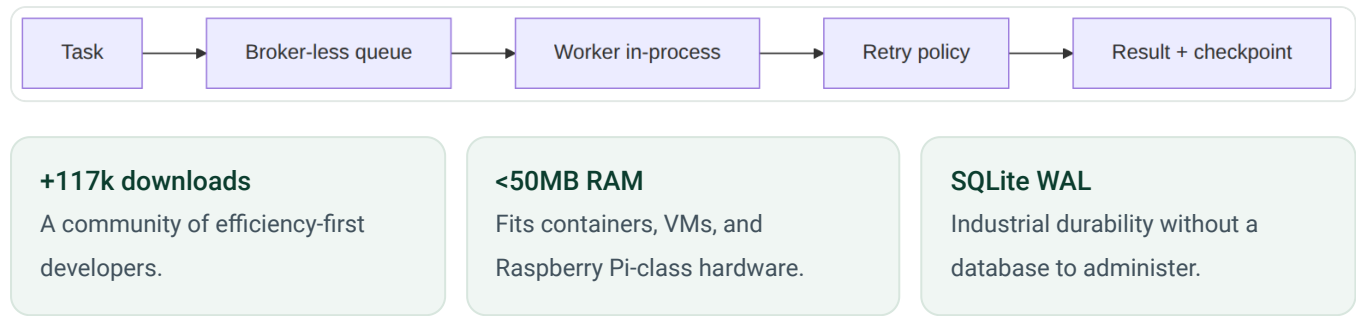
# Async-safe, but no Redis/RabbitMQ required.
```

## At a glance

| Metric | wpipe | Celery/Redis |
|--------|-------|--------------|
| Energy | Low   | High         |
| RAM    | <50MB | 250MB+       |
| Trust  | +117k | Standard     |

## Under the hood

The flow behind the post, as wpipe renders it from code.



## Next step

wpipe is a Python library — try it in your next pipeline:

```
pip install wpipe
```

Steps stay functions you already understand. Docs, diagrams, and dashboards follow. Explore **Celery** workflows at [wpipe.readthedocs.io](https://wpipe.readthedocs.io).

wpipe · Python-native orchestration with SQLite WAL checkpoints, retries, timeouts, parallelism, and self-documenting flows. +117k downloads.